



Pontificia Universidad Católica de Chile  
Escuela de Ingeniería  
Departamento de Ciencia de la Computación  
IIC2523 – Sistemas Distribuidos

## Tarea 1

### Comunicación y Sincronización en Sistemas Distribuidos

Fecha de inicio: **Miércoles 27 de agosto** a las **20:00:00** hrs

Plazo máximo para **formalizar parejas**: **Lunes 1 de septiembre** a las **20:00:00** hrs.

Formulario de inscripción: <https://forms.office.com/r/N9Lv3fuquf> Es obligatorio responder este formulario si trabajarás con otro estudiante. En otro caso, se asumirá que trabajarás de forma individual y no se aceptará reclamos al respecto.

Fecha de entrega: **Viernes 5 de septiembre** a las **20:00:00** hrs.

Fecha máxima de entrega (con atraso): **Domingo 7 de septiembre** a las **20:00:00** hrs.

### Evaluación en el contexto del curso

Esta es una evaluación **de corrección automática** y con la opción de realizarla de forma **individual** o **en parejas**. Además, pretende rescatar evidencias del desarrollo de ciertos resultados de aprendizaje. Por eso, tras su realización y entrega, recibirás retroalimentación respecto a los diferentes *tests* que se esperaba que la tarea lograra exitosamente. En particular, los objetivos específicos de esta evaluación son:

- Diseño e implementación de sistemas distribuidos con comunicación entre múltiples nodos.
- Manejo y sincronización de relojes lógicos de *Lamport* para resolver conflictos temporales.
- Propagación y consolidación de datos en sistemas distribuidos manteniendo consistencia.
- Desarrollo de interfaces RPC para gestión y consulta de datos en sistemas distribuidos.

## Contexto

Hace algunos días se celebró el grandioso concierto de la cantante Ado, el cual su profesor pudo disfrutar como nunca antes. Tras esa fascinante experiencia, conversó con muchas personas fanáticas de la cultura japonesa y del anime, quienes, entre canciones y recuerdos de series, protagonizaron encendidas discusiones sobre qué animes merecen un lugar en el Olimpo... y cuáles no deberían haber existido jamás. A partir de esas conversaciones, a su profesor se le ocurrió una idea brillante: ¿y si existiera una forma justa y ordenada de registrar todas esas opiniones apasionadas, sin que terminaran en una guerra de *gifs* y mayúsculas en redes sociales?

Así nació un interesante proyecto: un espacio donde la comunidad pudiera dejar sus calificaciones con confianza, sin temor a que desaparecieran misteriosamente. La meta era encontrar una manera en que todas las opiniones fueran consideradas con equilibrio, evitando el caos cada vez que alguien se atreviera a ponerle un 4 a ese anime que “todo el mundo debería amar”. Una cruzada por la justicia otaku, nacida de un concierto inolvidable.

Como el profesor estaba demasiado ocupado poniéndose al día con tantos animes y mangas, decidió delegar este maravilloso proyecto a sus estudiantes de Sistemas Distribuidos, quienes no tuvieron otra opción más aceptar el trabajo. Dado lo anterior, tu rol en esta ocasión consistirá en ayudar a esta comunidad desarrollando un pequeño sistema distribuido para calificar diferentes animes.

## Introducción

En esta evaluación desarrollarás un sistema distribuido compuesto por un servidor RPC y tres nodos con conexión TCP/IP que permiten registrar y gestionar calificaciones de animes hechas por diferentes usuarios. El objetivo es manejar correctamente la sincronización y propagación de datos entre nodos activos, incluyendo tolerancia a fallos y resolución de conflictos, garantizando que se mantenga una base de datos consolidada y confiable.

## 1. Servidor RPC para gestión de base de datos

En esta parte debes implementar un servidor RPC en Python que permita gestionar la calificación de animes por parte de los usuarios. Para lograr este objetivo, se proveerá de un código base (`rpc.py`) que deberás modificar y completar para esta parte de la evaluación **con Python 3.X con  $X \leq 12$** .

Este servidor RPC debe presentar, como **mínimo**, las siguientes funcionalidades:

1. `calificar(usuario: str, anime: str, puntaje: int) → int`.

Registra una calificación de un usuario a un anime. En caso de no existir el usuario o anime, este se crea por primera vez. Luego, si el usuario ya ha calificado previamente ese anime, la nueva calificación debe reemplazar la anterior. Retorna el puntaje almacenado en la base de datos.

2. `obtener(usuario: str, anime: str) → int`.

Consulta la calificación que un usuario ha asignado a un anime específico. Retorna el puntaje almacenado. En caso de no existir el usuario o el anime indicado, retorna `-1`.

3. `puntaje_promedio(anime: str) → float`.

Calcula el puntaje promedio de un anime. Retorna un *float* con el promedio redondeado a dos decimales (usar `round(puntaje, 2)` al momento de retornar). Si el anime no existe, retorna `-1`. Puedes asumir que en caso de existir el anime, tendrá al menos una calificación.

4. `animes() → dict`.

Retorna un diccionario donde la llave corresponde al nombre de un anime y el valor indica la cantidad de calificaciones que ha recibido. Si no existen animes en la base de datos, retorna un diccionario vacío.

Junto al servidor RPC, se provee de un archivo llamado `base_de_datos.json` que contiene una lista con las calificaciones realizadas a la fecha. Cada calificación es guardada como un diccionario cuyas llaves son: `usuario`, `anime` y `puntaje`. Cada vez que se realiza una calificación o se actualiza el puntaje de una, este archivo debe ser actualizado.

Un posible ejemplo `base_de_datos.json` es:

```
1 [
2   {"usuario": "Hernán V", "anime": "Lycoris Recoil", "puntaje": 10},
3   {"usuario": "Pablo", "anime": "Bungou Stray Dogs", "puntaje": 10},
4   {"usuario": "Gery", "anime": "Blue Lock", "puntaje": 10}
5 ]
```

Puedes asumir que este archivo estará siempre disponible y ubicado en la misma carpeta que `rpc.py`. No obstante, es posible que al inicio se encuentre vacío, es decir, contenga simplemente una arreglo vacío (`[]`). Además, en caso de incluir calificaciones, estas estarán correctamente formateadas desde el principio. A medida que realices modificaciones en el archivo, será tu responsabilidad asegurar que el formato se mantenga correcto.

Finalmente, se permite incorporar nuevas funcionalidades al RPC, siempre que sea para **uso interno** de la tarea.

## 2. Sistema Distribuido con Nodos TCP/IP

En esta sección implementarás tres nodos encargados de gestionar las calificaciones que los usuarios asignan a distintos animes. Cada nodo mantendrá una copia local de los datos y utilizará un reloj lógico de *Lamport* para sincronizarse y resolver conflictos. Para ello, se te proporcionará un código base (`servidor.py`) que deberás modificar y completar como parte de esta evaluación **con Python 3.X con  $X \leq 12$** . Este archivo simula un solo nodo, por lo que deberá ejecutarse tres veces en paralelo para representar los tres nodos.

Este sistema distribuido de tres nodos debe cumplir con los siguientes requisitos:

- Cada nodo debe comunicarse utilizando el protocolo TCP/IP a través de `sockets` en Python.
- Los nodos utilizarán un reloj lógico de *Lamport* para registrar los eventos y la sincronización entre ellos. Queda a tu criterio cómo implementarlo y comunicarlo internamente entre los nodos.
- Al recibir una calificación del usuario, el nodo realiza 2 eventos antes de procesar el siguiente mensaje: primero registra la información de forma local y luego la propaga a los demás nodos. Un nodo que recibe la calificación debido a la propagación de otro nodo no realiza la propagación nuevamente, solo gestiona el registro de la información localmente.
- El reloj de *Lamport* se actualiza en tres situaciones: al momento que el nodo recibe una calificación del usuario, al momento de propagar una calificación y cuando se recibe un mensaje de otro nodo. Respecto a la propagación de una calificación, este cuenta como un solo evento sin importar la cantidad de receptores.
- Ante la recepción de una calificación o de empezar la acción de propagar, primero se debe actualizar el reloj lógico del nodo y luego realizar la acción de almacenar la calificación o propagar la información.
- Cada calificación consiste en un puntaje entero entre 1 y 10, asignado por un usuario a un anime. Cada usuario puede calificar un mismo anime solo una vez. En caso de que un usuario envíe una nueva calificación para un anime que ya ha calificado, se debe almacenar el puntaje de la nueva calificación.
- Existen dos tipos de mensajes destinados a simular una falla temporal en el nodo:
  - Al recibir un mensaje del tipo '**STOP**', el nodo entra en un estado inactivo, en el cual no procesa calificaciones ni propaga información a otros nodos. Dado lo anterior, su reloj de *Lamport* tampoco se ve actualizado mientras esté en este estado.
  - Al recibir un mensaje del tipo '**START**', el nodo vuelve a estar activo. Si el nodo ya se encontraba activo, este mensaje no tendrá ningún efecto ni generará errores durante la ejecución.
- Puedes asumir que siempre se levantarán los 3 nodos, es decir, ejecutar tres veces `servidor.py`, antes de que se empiecen a enviar los mensajes de algún nodo.
- Toda la comunicación entre los nodos debe ser mediante el envío de mensajes simulando que cada elemento se encuentra en computadores separados. **No se aceptarán decisiones que funcionen exclusivamente porque los nodos están en un mismo computador.**

## 2.1. Consolidación de datos en servidor RPC

Una funcionalidad muy importante es que los nodos incluyen un **proceso de consolidación de datos**. Este proceso consiste en unificar las calificaciones locales de cada nodo para todos tengan la misma información. Además, una vez finalizado el proceso de consolidar la información, se debe enviar el resultado de dicha consolidación al servidor RPC para que este también quede actualizado.

El mecanismo para implementar este proceso debe cumplir las siguientes condiciones:

- En cualquier momento, un nodo activo puede recibir un mensaje del tipo '**CONSOLIDAR**'. Al hacerlo, dicho nodo inicia este proceso para unificar la información entre los demás nodos activos y actualizar el RPC.
- Si un nodo inactivo recibe un mensaje del tipo '**CONSOLIDAR**', no inicia el proceso de consolidación.
- La consolidación solo debe considerar la información proveniente de los nodos que se encuentren activos al momento de iniciar el proceso.
- Al momento de consolidar, si múltiples nodos almacenan diferentes calificaciones de un usuario para un mismo anime, el sistema debe conservar únicamente la calificación registrada más recientemente según el reloj lógico de dicho evento. En caso de empate entre los relojes, la calificación se considerará inconsistente, así que se mantendrá únicamente la con mayor puntaje.
- Una vez finalizado el proceso de consolidación, el reloj lógico de todos los nodos participantes debe reiniciarse.
- Se debe utilizar correctamente `xmlrpc.client.ServerProxy` de Python para la comunicación de los nodos con el servidor RPC.
- La actualización del RCP posterior a la consolidación solo debe agregar y actualizar las calificaciones que tenga el RCP. En caso que el RCP contenga una o más calificaciones que los nodos desconocían, estas no se deben perder tras el proceso de consolidación.
- Toda la comunicación de los nodos con el servidor RPC debe ser mediante el envío de mensajes simulando que cada se encuentra en computadores separados. **No se aceptarán decisiones que funcionen exclusivamente porque el servidor RPC y los nodos están en un mismo computador.**

## 2.2. Formato mensajes TCP

Cada mensaje que llega a algunos de los nodos corresponderá primero a **4 bytes** en formato *big-endian* que indicarán el tamaño del mensaje que llegará posteriormente. Luego, los siguientes **N bytes** corresponderán a un diccionario que fue codificado con `json` y en `UTF-8`<sup>1</sup>. Este diccionario posee las siguientes 4 llaves:

- **tipo**. En esta llave se indica el tipo de mensaje a enviar, el cual puede ser:
  - **'SCORE'**: Para enviar una calificación (debe incluir usuario, anime y puntaje).
  - **'STOP'**: Indica que el nodo debe entrar en estado inactivo.
  - **'START'**: Indica que el nodo debe volver a estar activo.
  - **'CONSOLIDAR'**: Solicita la consolidación entre los nodos.
- **usuario**: En esta llave se incluye el nombre del usuario. El nombre de usuario será un *string* cuyo largo es de 1 a 40 caracteres alfanuméricos y puede contener espacios. Esta llave solo está si el mensaje es del tipo **'SCORE'**.
- **anime**: En esta llave se incluye el nombre del anime a clasificar. El nombre de usuario será un *string* cuyo largo es de 1 a 100 caracteres alfanuméricos y puede contener espacios. Esta llave solo está si el mensaje es del tipo **'SCORE'**.
- **puntaje**: En esta llave se incluye el puntaje dado al anime. El puntaje será un *int* que solo podrá tomar valores entre 1 y 10 inclusive. Esta llave solo está si el mensaje es del tipo **'SCORE'**.

Puedes asumir que los diccionarios siempre respetarán el formato indicado en el enunciado y las restricciones para cada llave, por lo cual, no es necesario validar el formato. A continuación se muestran algunos ejemplos de diccionarios que se pueden enviar:

```
1 {"tipo": 'SCORE', "usuario": "Hernán V", "anime": "Lycoris Recoil", "puntaje": 10}
2 {"tipo": 'SCORE', "usuario": "Pablo", "anime": "Bungou Stray Dogs", "puntaje": 10}
3 {"tipo": 'STOP'}
4 {"tipo": 'START'}
5 {"tipo": 'CONSOLIDAR'}
6 {"tipo": 'SCORE', "usuario": "Hernán V", "anime": "Gintama", "puntaje": 10}
7 {"tipo": 'SCORE', "usuario": "Gery", "anime": "Haikyuu", "puntaje": 10}
8 {"tipo": 'SCORE', "usuario": "Miku", "anime": "Tengen Toppa Gurren Lagann", "puntaje": 4}
```

El mecanismo y formato utilizados para la comunicación interna entre nodos —tanto en la propagación como en la consolidación de información— quedan a criterio de cada uno mientras se respeten las reglas del enunciado. Además, debes asegurar únicamente que **dicho mecanismo no demore más de 0.2 segundos en completarse**. Este tiempo (0.2 segundos) será el intervalo que cada *test* utilizará para esperar entre el envío de un mensaje y el siguiente. La única excepción a esta regla es el mensaje del tipo **'CONSOLIDAR'**, donde el *test* podrá a esperar hasta 5 segundos antes de enviar el siguiente mensaje.

---

<sup>1</sup>Puedes leer la función `enviar_mensaje` del archivo `test_servidor.py` para ver con más detalle cómo se envía el mensaje desde el usuario

### 2.3. Ejemplos de calificación y consolidación

A continuación se presentan algunos ejemplos de llamados a los distintos nodos. Para simplificar, cada ejemplo asume que la base de datos estaba inicialmente vacía.

#### Ejemplo 1 - *Rating* se sobrescribe

1. Sakura califica Detective Conan con 7 en Nodo A.
2. Keni califica One Piece con 10 en Nodo B.
3. Sakura califica Detective Conan con 9 en Nodo C.
4. 'CONSOLIDAR' a Nodo B.

La base de datos final será:

- Sakura / Detective Conan / 9
- Keni / One Piece / 10

#### Ejemplo 2 - *Rating* se sobrescribe por nodo que tiene menor reloj lógico

1. 'STOP' a Nodo C.
2. Sakura califica Detective Conan con 6 en Nodo B.
3. Keni califica One Piece con 8 en Nodo B.
4. Ruby califica Gintama con 7 en Nodo B.
5. 'START' a Nodo C.
6. Ruby califica Gintama con 9 en Nodo C.
7. 'CONSOLIDAR' a Nodo A.

La base de datos final será:

- Sakura / Detective Conan / 6
- Keni / One Piece / 8
- Ruby / Gintama / 9

#### Ejemplo 3 - Información enviada a un nodo caído no se considera

1. Aqua califica Your Name con 5 en Nodo A.
2. 'STOP' a Nodo B.
3. Miku califica Your Name con 7 en Nodo B.
4. Miku califica Lycoris Recoil con 10 en Nodo A.
5. 'CONSOLIDAR' a Nodo C.

La base de datos final será:

- Aqua / Your Name / 5
- Miku / Lycoris Recoil / 10

#### **Ejemplo 4 - Se pierde información por nodos apagados**

1. 'STOP' a Nodo A.
2. 'STOP' a Nodo B.
3. Sakura califica Detective Conan con 7 en Nodo C.
4. 'START' a Nodo A.
5. 'START' a Nodo B.
6. 'START' a Nodo C.
7. 'STOP' a Nodo C.
8. Keni califica One Piece con 5 en Nodo A.
9. Ruby califica Gintama con 9 en Nodo B.
10. 'CONSOLIDAR' a Nodo B.

La base de datos final será:

- Keni / One Piece / 5
- Ruby / Gintama / 9

#### **Ejemplo 5 - Se pierde información por no consolidar**

Dada las siguientes consultas:

1. Sakura califica Detective Conan con 9 en Nodo C.
2. 'CONSOLIDAR' a Nodo B.
3. Keni califica One Piece con 5 en Nodo A.
4. Ruby califica Gintama con 9 en Nodo B.

La base de datos final será:

- Sakura / Detective Conan / 9



### 3. Archivos

Para el desarrollo de esta evaluación, se provee de los siguientes archivos:

- **Modificar** `rpc.py`

Este es el archivo que se ejecutará para levantar el servidor RPC.

El formato para ejecutar este archivo es `python3 rpc.py [PUERTO_RPC]`, donde `PUERTO_RPC` corresponde al puerto que utilizará el servidor RPC.

Un ejemplo de ejecución es `python3 rpc.py 8711`.

- **Modificar** `servidor.py`

Este es el archivo que se ejecutará para levantar un nodo TCP. Puedes crear más archivos si deseas modularizar, pero al momento de corregir, este será el archivo principal que debe ejecutarse para cada nodo.

El formato para ejecutar este archivo es `python3 servidor.py [A-B-C] [PUERTO_A] [PUERTO_B] [PUERTO_C] [PUERTO_RPC]`, donde `[A-B-C]` es un *string* que identifica el nodo a ejecutar: 'A', 'B' o 'C'. Luego, `PUERTO_A`, `PUERTO_B` y `PUERTO_C` corresponden a los puertos asignados a los nodos A, B y C respectivamente, y `PUERTO_RPC` es el puerto asignado al servidor RPC.

Un ejemplo de ejecución es `python3 servidor.py C 4444 4445 4446 8711`.

- **Modificar** `constantes.py`

Archivo para definir la constante `COMANDO_PYTHON`. Esta constante será utilizada por los archivos `test_servidor.py` y `test_rpc.py` para determinar el comando necesario para ejecutar archivos Python en el computador del estudiante. **Se prohíbe agregar más constantes a este archivo**, solo debes actualizar la constante de `COMANDO_PYTHON` según corresponda.

- **No modificar** `base_de_datos.json`

Este archivo contiene una lista inicial de calificaciones realizadas. En caso de no existir calificaciones, solo contendrá una lista vacía. Este archivo solo se debe ir actualizando según las consultas que lleguen al servidor RPC.

- **No modificar** `test_rpc.py`

Este es un archivo de ejemplo para probar diferentes *test case* de la parte del sistema RPC. Se encargará de buscar un puerto libre, levantar el sistema RPC y ejecutar una secuencia de consultas. Durante la corrección, este archivo será reemplazado por otro archivo Python cuyo contenido y forma de probar los *tests* puede cambiar, pero se respetará la forma de ejecutar y comunicarse con el RPC creado en `rpc.py`.

Para ejecutar este archivo, solo hay que hacer `python3 test_rpc.py`

- **No modificar** `test_servidor.py`

Este es un archivo de ejemplo para probar diferentes *test case* para el sistema distribuido. Se encargará de buscar cuatro puertos libres, levantar el sistema RPC junto a los tres nodos y ejecutar una secuencia de consultas. Durante la corrección, este archivo será reemplazado por otro archivo Python cuyo contenido y forma de probar los *tests* puede cambiar, pero se respetará la forma de ejecutar y comunicarse con el servidor distribuido creado en `servidor.py`.

Para ejecutar este archivo, solo hay que hacer `python3 test_servidor.py`

- **No modificar** `tests/`

Carpeta utilizada por `test_servidor.py` y `test_rpc.py` con diversos *tests case*.

## 4. Corrección automatizada

La corrección de toda la evaluación será **únicamente mediante test cases** para evaluar diferentes casos borde que puedan existir. En consecuencia, entregar una evaluación que no pueda ejecutarse implicará que esta será calificada con la nota mínima y sin posibilidad de corregir el error. Es responsabilidad del estudiante entregar un código que se pueda ejecutar correctamente en consola.

Adicionalmente, **no se permite utilizar ninguna librería adicional a las ya incluidas al inicio de los archivos entregados**. Si se detecta su uso, dicha tarea será evaluada con nota mínima.

Respecto a la corrección del sistema RPC, algunos *tests* realizarán diferentes consultas de lectura (`obtener`, `puntaje_promedio` y `animés`) a partir de una base de datos con 0, 1 o más calificaciones. Luego, otro conjunto de *tests* realizarán diversas consultas de escritura (`calificar`) junto con varias consultas de lectura que verificarán la correcta actualización de la base de datos. Se provee del archivo `test_rpc.py` con un ejemplo de algunas consultas de lectura y escritura.

Posteriormente, para la corrección del sistema distribuido, se efectuarán consultas a los nodos A, B y C para calificar distintos animés y consolidar los datos en el sistema RPC. Luego, se verificará el archivo `base_de_datos.json` para comprobar que los datos estén correctamente consolidados. Se provee del archivo `test_servidor.py` con un ejemplo de algunas calificaciones y la verificación de la base de datos.

Dado lo anterior, es **imperativo** asegurar que las consultas de lectura y actualización de la base de datos en el RPC estén correctamente implementadas para que los *test cases* finalicen exitosamente.

Es importante recordar que los **tests públicos son solo un subconjunto de todos los casos que se evaluarán**. Queda a responsabilidad del estudiante probar la tareas con otros casos para validar la correcta implementación del sistema distribuido y el servidor RPC.

### Desglose de puntaje

- **(0.5 puntos)** Los métodos de consulta de datos del RPC (`obtener`, `puntaje_promedio`, `animés`) funciona correctamente a partir de una base de datos ya existente.
- **(0.5 puntos)** Los métodos de consulta de datos del RPC (`obtener`, `puntaje_promedio`, `animés`) se actualiza correctamente tras calificar uno o más animés con el método `calificar` del RPC.
- **(1.0 punto)** El sistema distribuido pasa los *tests* con casos donde todos los nodos están activos en todo momento. En estos casos pueden haber una o más consolidación de datos.
- **(1.0 punto)** El sistema distribuido pasa los *tests* con casos donde uno o más nodos se desactivan permanentemente (es decir, reciben `STOP` pero no `START`). En estos casos pueden haber una o más consolidación de datos.
- **(1.5 puntos)** El sistema distribuido pasa los *tests* con casos donde uno o más nodos se desactivan temporalmente (es decir, reciben `STOP` y luego `START`) antes de consolidar los datos. En estos casos, cada *tests* tendrá máximo una consolidación de datos que se logra efectuar exitosamente con los nodos activos.
- **(1.5 puntos)** El sistema distribuido pasa los *tests* con casos donde uno o más nodos se desactivan temporalmente y se realiza más de una consolidación de datos entre los nodos activos.

## Asignación de puntaje

Para la asignación de puntaje, cada *test* podrá recibir uno de los siguientes valores:

- **2 puntos:** Todas las consultas del *test* que verifican información son respondidas correctamente con el valor esperado.
- **1 punto:** Al menos el 60 % de los consultas del *test* son respondidas correctamente con el valor esperado.
- **0 puntos:** No se alcanza el porcentaje de respuestas correctas esperadas en las consultas del *test*.

En caso de que algún *test* realice solo una consulta, el puntaje será binario: 2 puntos si la única consulta está correcta o 0 puntos en otro caso.

Finalmente, cada ítem estará compuesto por un conjunto arbitrario de *tests*. Sea  $N$  la cantidad total de *tests* para un ítem, se calculará el nivel de logro del ítem, el cual se utilizará para ponderar su puntaje máximo y así obtener el puntaje final correspondiente. La fórmula será la siguiente:

$$puntaje\_obtenido\_item_x = round \left( \frac{total\_puntaje\_tests\_logrado_x}{N \times 2} \times puntaje\_maximo\_item_x, 2 \right)$$

## Descuentos

La evaluación está sujeta a ciertas reglas formales. El incumplimiento de alguna de ellas puede implicar descuentos automáticos o a criterio del equipo docente, según se indica a continuación:

### ■ PEP8 – Largo de línea

Cada línea del archivo debe tener un **máximo de 100 caracteres**, incluyendo espacios y tabulaciones. Si al menos **una línea** supera los 100 caracteres (es decir, tiene 101 o más), se descontarán 0.2 puntos de la nota final, incluso si la línea excedente contiene solo espacios o tabulaciones.

### ■ Modularización

Cada archivo `.py` puede contener un **máximo de 400 líneas**, incluyendo código, comentarios y líneas en blanco. Si al menos **un archivo supera este límite** (401 líneas o más), se descontarán 0.2 puntos de la nota final, sin importar el contenido de la línea extra.

### ■ Formato de Entrega

Si el estudiante entrega una solución funcional, pero esta no respeta alguna regla del enunciado o del curso, el descuento será determinado por el equipo docente, y podrá variar entre 0.1 puntos y hasta 6.0 puntos, dependiendo de la gravedad del error.

Por ejemplo, si un estudiante utiliza una librería externa que no estaba permitida y debido a ello, el código no se ejecuta correctamente en el computador del equipo docente, la penalización será de 6.0 puntos.

En cambio, si el estudiante incluye un `import` de una librería no permitida que impide la ejecución del código en los computadores del equipo docente, pero el código no hace uso real de dicha librería y solamente eliminando el `import` permite que el código funcione. El equipo docente puede evaluar la posibilidad de eliminar dicho `import` para ejecutar la tarea, pero se aplicaría un descuento según lo definido anteriormente.

## 5. Entregables

Se espera que el entregable sea **un archivo ZIP** que contenga los archivos **rpc.py** y **servidor.py**. En caso de crear archivos Python adicionales para modularizar la tarea, estos también deben estar incluidos en el ZIP.

El archivo `.zip` entregado también debe incluir un archivo `README.md` que contenga:

- Todas las **citas** de material externo utilizado.
- Una explicación clara y completa del uso de cualquier **herramienta generadora de código** (por ejemplo: asistentes de inteligencia artificial, generadores automáticos, etc.).

Si el archivo `README.md` está ausente o no incluye alguna de las referencias necesarias, se asumirá que **todo el contenido entregado sin referencias es de autoría exclusiva del estudiante**.

**Advertencia:** Si se detecta uso de material externo o herramientas generativas sin que estén debidamente citadas en el `README.md`, se considerará una **falta a la integridad académica**, y se aplicarán las sanciones correspondientes según las políticas establecidas en el programa del curso.

En caso de trabajar en parejas, solo un integrante debe entregar el trabajo. En caso que ambos entreguen, se revisará cualquiera de los 2 entregas sin poder apelar a dicha decisión.

Finalmente, no se aceptarán entregas en formatos distintos al indicado (por ejemplo, archivos `.ipynb`, `.pyc`, etc.). Si no se entrega el archivo ZIP o se entrega en un formato diferente, la entrega no será revisada y se asignará la nota mínima.

## 6. Dudas

Cualquier duda que tengas sobre esta evaluación, prefiere publicarla en el [Syllabus](#) correspondiente a esta evaluación. También, siente la libertad de responder dudas de tus pares si crees que conoces la respuesta. En caso de tener dudas que impliquen mostrar tu solución o partes de ella, no utilices este medio de consulta. Para estos casos, envía un correo al cuerpo docente o muestra tu solución solo en reunión personal (remota o presencial) cuando te reúnas con algún miembro del cuerpo docente.

## 7. Política de atraso

Existe la posibilidad de entregar esta evaluación con hasta **2 días de atraso** a partir de la fecha de entrega definida en el enunciado. En la eventualidad de entregar pasada la fecha de entrega, se aplicará una **reducción** a la nota máxima que podrás obtener en esta evaluación.

Si el trabajo se realiza en parejas, se corregirá aquella entregada con atraso y la reducción de nota aplicará a ambos miembros.

De haber atraso, **la nota máxima a obtener** se reduce en **1 punto (10 décimas)** por cada día de atraso. Cada día de atraso se determina como el techo de días de atraso. Por lo tanto, en caso de entregas atrasadas, la nota final de la tarea se calcula mediante la siguiente fórmula:

$$\text{mín}(7 - (0.5 \times \text{días\_atraso}), \text{nota\_obtenida}) - \text{descuento\_total})$$

Por otro lado, entregas con más de 2 días (48 hrs) de atraso no serán recibidas y serán evaluadas con la **calificación mínima (1.0)**.

## 8. Integridad académica y código de honor

Recuerde que esta evaluación está alineada al Código de Honor de la Universidad y se espera un comportamiento íntegro del estudiantado al momento de desarrollar. Dado lo anterior, se **enfatisa** que la solución de esta evaluación debe ser producto de un máximo de 2 estudiantes que trabajaron en conjunto y se inscribieron oportunamente en el formulario indicado al inicio de este enunciado.

Finalmente, respecto al uso de herramientas generadoras de código. Estas están permitidas siempre y cuando se referencie correctamente cuándo es utilizado. Junto a lo anterior, se espera que **toda respuesta entregada por una herramienta generadora de código o de IA pase por un proceso de análisis crítico y modificación antes de ser incluida en una evaluación**. En caso de entregar una tarea donde se identifique que solo hubo un proceso de copiar la respuesta dada por una herramienta generadora de código o de IA, esto no será aceptado y será considerado como una falta a la integridad académica.